

CS3002E Compiler Design

Monsoon 2025

Lecture #21

Semantic Analysis - Symbol Table

Department of Computer Science & Engineering
NIT Calicut

Compiler: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    y = 0;
}
```

error: 'y' undeclared ...

Is this a Syntax error (reported by the parser)?

Compiler: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    x[1]=0;
}
```

error: subscripted value is neither array nor ...

Which phase detects this error?

Compile time checks

- Syntax Analysis checks if the program is syntactically valid and reports syntax errors, if any.
- There are program errors that can not be detected during syntax analysis.

Error Detection

Examples of errors that can not be detected during syntax analysis:

- Type mismatch
- Variable used without declaration
- Array access out of bounds
- Divide by zero

Is it possible to detect these errors **statically** (during compilation)?

Static checks

Examples of errors that can be detected statically (through **static** checks):

- Type mismatch
- Variable used without declaration

Checks like *array out of bounds* require dynamic information.

Semantic Analysis

Semantic Analysis: checks the source program for semantic consistency with the language definition. Main jobs are:

- Type checking (major part of semantic analysis): checks if each operator is applied to the right type of operands.
- Some of the checks that can not be done during syntax analysis (e.g. whether a variable is declared before use).
- Type conversions (coercions) as per the type conversion rules of the language.

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    y=0;
}
```

error: 'y' undeclared ...

Error Detection: How? Required information ?

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    y=0;
}
```

error: 'y' undeclared ...

For error detection:

- Compiler has to keep a list of all variables that are declared.
- Which phase creates this list?

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    y=0;
}
```

error: 'y' undeclared ...

Error Detection:

- The parser while parsing the declaration section can create a list of all variables that are declared.
- For each use of a variable, compiler can check if the variable has an entry in the list.

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    ...
    x[1]=0;
}
```

error: subscripted value is neither array nor ...

Error detection?

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    ...
    x[1]=0;
}
```

error: subscripted value is neither array nor ...

For error detection:

- Compiler has to remember the type of each variable.
 - Maintain the list of declared variables, along with their type.
- Which phase stores the type information?

Symbol Table

Symbol Table

- A data structure that maintains information regarding the source program.
- One entry for each name (*symbol*) - names of variables, symbolic constants, labels, procedure/function names etc., with values of its attributes.
- Stores information collected during initial phases of *Analysis*, that will be required for later stages of compilation.

Symbol Table

- Possible attributes for names:
 - variable name - name (lexeme), type, scope, binding (relative position in storage).
 - function name - name, number of arguments, name and type of each argument, the method of passing each argument, return type.

Example: Symbol Table entries created while processing the declaration `int x; float y;`

Name	Type	...
x	int	...
y	float	...
...	...	...

Symbol Table: Implementation

- An implementation of the *Dictionary* ADT (operations: *insert*, *lookup* and *delete*)
 - list, search trees, hash tables (commonly used for efficiency reasons).
- Each entry is a record containing the values of the various attributes.

Semantic Analysis

Semantic Analysis: checks the source program for semantic consistency with the source language.

- uses the Intermediate Representation (IR) generated by the parser and the information in the Symbol Table.
- does **Type Checkings**, **Type Conversions** and static checks that can not be done by the syntax analyzer.

Semantic Analysis

- **Type Checking:** an important part of semantic analysis- checks if each operator is applied to operands of expected types. For example,
 - if the operands of an integer addition are of integer type
 - if the number and types of arguments in a function call match with the function prototype
- **Type Conversions:** According to the typing rules of the language.
 - e.g. in the C program fragment:
`int x; float y; ....x+y...`
the type of x in expression x+y is converted to float.
- static checks that can not be done by the syntax analyzer -
e.g. checking if a variable is declared before its use.

Symbol Table Management

Symbol Table Management: Lexical Analysis

```
#include <stdio.h>
int main()
{
    int x; float y;
    ...
}
```

- What are the tokens returned?
- What is the attribute for each *id* token?
- Can Symbol Table (ST) entries for *x* and *y* be created by the Lexical Analyzer?

Lexical Analysis: Symbol Table entries

```
#include <stdio.h>
int main()
{
    int x; float y;
    ...
}
```

Lexical Analyzer:

- Creates a Symbol Table entry for each identifier.
- Token returned: *< id, pointer to ST entry >*

Lexical Analysis

Input: `int x; float y;`

Symbol Table entries:

Name	Type	...
x	...	...
y	...	...
...	...	...

Token Stream:

`< int >` `< id, pointer to ST entry for x >` `< semicolon >`
`< float >` `< id, pointer to ST entry for y >` `< semicolon >`

Lexical Analysis: Symbol Table entries

```
#include <stdio.h>
int main()
{
    int x; float y;
    ...
    x = x+y;
}
```

How many ST entries are created by the Lexical Analyzer?

Lexical Analysis: Symbol Table entries

```
#include <stdio.h>
int main()
{
    int x; float y;
    ...
    x = x+y;
}
```

How many ST entries created by the Lexical Analyzer?

- One entry for each identifier, created upon seeing the first occurrence of that identifier.
- For each occurrence of an identifier, a Symbol Table lookup is done to check if it is already entered.

Syntax Analysis: Symbol Table updation

```
#include <stdio.h>
int main()
{
    int x; float y;
    ...
}
```

Parser while parsing the declaration section:

- sets the type of each identifier in the ST.
- the attribute value in the identifier token is used for finding the corresponding ST entry.

Syntax Analysis: Input

Token Stream:

$\langle \text{int} \rangle \langle \text{id, pointer to ST entry for } x \rangle \langle \text{semicolon} \rangle$
 $\langle \text{float} \rangle \langle \text{id, pointer to ST entry for } y \rangle \langle \text{semicolon} \rangle$

Grammar Rules:

$D \rightarrow T L$

$T \rightarrow \text{int} \mid \text{float}$

$L \rightarrow L, \text{id} \mid \text{id}$

SDD for setting types in Symbol Table

Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \mathbf{int}$	$T.type = int$
$T \rightarrow \mathbf{float}$	$T.type = float$
$L \rightarrow L_1, \mathbf{id}$	$L_1.type = L.type$ $addType(\mathbf{id}.entry, L.type)$
$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.type)$

Attribute $L.type$ is *inherited*.

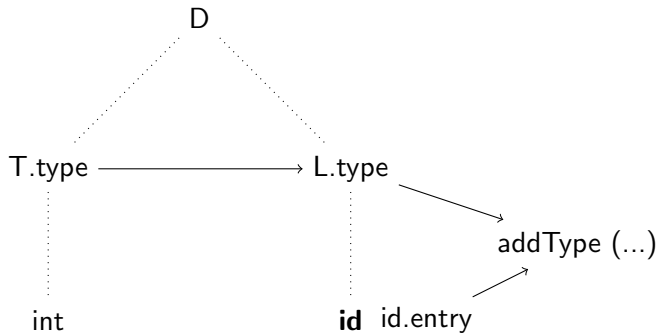
- Semantic rule $L.type = T.type$ associated with $D \rightarrow T L$
 $L.type$ is dependent on the attribute at its sibling node.
- Semantic rule $L_1.type = L.type$ associated with $L \rightarrow L_1, \mathbf{id}$
 $L_1.type$ is dependent on the attribute at its parent node.

SDD for setting types in Symbol Table

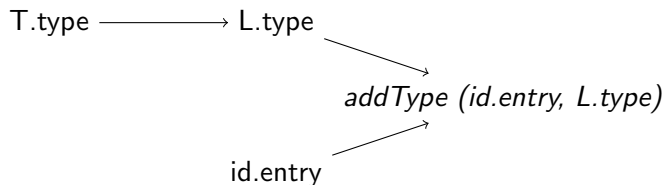
Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \mathbf{int}$	$T.type = int$
$T \rightarrow \mathbf{float}$	$T.type = float$
$L \rightarrow L_1, \mathbf{id}$	$L_1.type = L.type$ $addType(\mathbf{id}.entry, L.type)$
$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.type)$

- $\mathbf{id}.entry$ is a pointer to the Symbol Table entry for $\mathbf{id}$. For each identifier, the value of $entry$ is assumed to be obtained from the Lexical Analyzer.
- $addType(id.entry, L.type)$ is invoked to set the value of type as $L.type$ in the Symbol Table entry pointed to by $id.entry$.

Dependency among attributes



Setting types in Symbol Table



- Attributes evaluated in a topological sort order of the dependency graph.
- *addType (id.entry, L.type)* sets the value of type as *L.type* in the ST entry pointed to by *id.entry*.

Exercise

Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \mathbf{int}$	$T.type = int$
$T \rightarrow \mathbf{float}$	$T.type = float$
$L \rightarrow L_1, \mathbf{id}$	$L_1.type = L.type$ $addType(\mathbf{id}.entry, L.type)$
$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.type)$

Exercise: Draw the annotated parse tree for *int id₁, id₂, id₃*

Exercise

Production	Semantic Rules
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow \mathbf{int}$	$T.type = int$
$T \rightarrow \mathbf{float}$	$T.type = float$
$L \rightarrow L_1, \mathbf{id}$	$L_1.type = L.type$ $addType(\mathbf{id}.entry, L.type)$
$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.type)$

Exercise: Draw the dependency graph for *int* id_1 , id_2 , id_3 . Add a dummy attribute node for each invocation of $addType()$ function. Find a possible order of evaluation of attributes.

Syntax Analysis: Updated Symbol Table

Name	Type	Size	Relative Address	...
x	int	4	...	...
y	float	8	...	...
...	...	...	...	...

Attributes *Size* and *Relative Address* are required during code generation.

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    ...
    y=0;
}
```

error: 'y' undeclared ...

Detecting use of an undeclared variable:

- For each declared variable, the parser has updated the ST entry with its type information.

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    ...
    y=0;
}
```

error: 'y' undeclared ...

Semantic Analysis, detecting use of an undeclared variable:

- For each use of an identifier, checks if the identifier has an entry in the Symbol Table.
- If there is no entry for the identifier or type of identifier is not set, reports error.

Semantic Analysis: Error Detection

```
#include <stdio.h>
int main()
{
    int x;
    float y;
    ...
    x[1]=0;
}
```

error: subscripted value is neither array nor ...¹

Error detected during Semantic Analysis - looks up the Symbol Table entry for x. Finds its type as int, not array.

¹Subscripting is permitted only on array type variables (as per the typing rules of the language).

Symbol Table: Implementing Scope

Scope

Scope of a declaration of a variable x is the portion of the program in which uses of x refer to this declaration.

- Static scope / Lexical Scope - possible to determine the scope of a declaration statically - C, Java use static scope.
- Dynamic scope - Scope can change dynamically (during runtime).

Scope: block structured Languages

A **block** is a grouping of declarations and statements

- The scope of a declaration of name x in a block B is all of B except for any blocks nested within B in which x is redeclared.
- Nested blocks - identifier x is in the scope of the most-closely nested declaration of x (*most-closely nested rule*).

Scope: Example - C code fragment

```
int x, y; //global x
...
void f(...){
    int x; //local x
    ...
    x=3; //use of local x
    ...
}
...
x=x+1; //use of global x
```

Block structured Languages: Symbol Table

- Maintain separate Symbol Tables - each block has its own symbol table with an entry for each declaration in that block.
- Table for a nested block points to the table for its enclosing block (*Chaining Symbol Tables*).
- *lookup* by searching the chain of symbol tables, starting with the table for the current block.
- Chaining results in a tree structure, since more than one block can be nested inside an enclosing block.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007

Further reading:

- ALSU Section 1.2, 1.6, Chapter 2 - Sections 2.7 - 2.7.1, Chapter 5- Section 5.2, Chapter 6 - Section 6.3 - 6.3.1, 6.3.2